

Day 60: Final Beginner Python Mini Project 🎉

1. Day Number

Day 60 of your Beginner Python Foundation Journey

You have reached the final day of your 60-day Python foundation track.

2. Topic Name

Final Revision Mini Project — Simple Bank App

Today you will combine the main Python concepts you have learned into one small program.

3. Connection

During the last 60 days, you gradually moved from basic Python statements to functions, files, error handling, and object-oriented programming.

Your progression was roughly:

```
Variables
  ↓
Input / Output
  ↓
Conditions
  ↓
Loops
  ↓
Strings
  ↓
Lists / Dictionaries
  ↓
Functions
  ↓
Error Handling
  ↓
Files
  ↓
Classes and Objects
  ↓
BankAccount methods
  ↓
Manual Testing
  ↓
Final Mini Project
```

Today you will bring those ideas together in a **very small bank application**.

The goal is not to create a real banking system. The goal is to prove that you can connect your Python fundamentals together.

4. Revision Summary of the Full 60 Days

Your 60-day journey covered several important stages.

Stage 1 — Basic Python

You started with things such as:

```
name = "Rahul"  
age = 25  
balance = 1000
```

You learned that variables store information.

You also learned output:

```
print(balance)
```

and user input:

```
name = input("Enter name: ")
```

Stage 2 — Type Conversion

`input()` normally gives you a string.

For calculations, you often convert it.

Example:

```
amount = float(input("Enter amount: "))
```

or:

```
age = int(input("Enter age: "))
```

Stage 3 — Conditions

You learned how programs make decisions.

```
if age >= 18:  
    print("Adult")  
else:  
    print("Minor")
```

Later you used:

```
if  
elif  
else
```

for multiple choices.

Stage 4 — Loops

Loops allow repeated work.

You practiced ideas such as:

```
for item in items:
    print(item)
```

and:

```
while condition:
    # repeat something
```

Loops became especially useful when working with lists and dictionaries.

Stage 5 — Strings

Strings represent text.

You practiced operations such as:

```
name.lower()
name.upper()
name.strip()
```

You also compared strings:

```
if password == "1234":
```

That idea will appear again in today's project.

Stage 6 — Lists

Lists store multiple values.

```
transactions = []
```

You learned to add items:

```
transactions.append("Deposit")
```

and loop through them:

```
for transaction in transactions:
    print(transaction)
```

Today's project uses a list for a transaction.

Stage 7 — Dictionaries

Dictionaries store information using **key-value pairs**.

Example:

```
product = {  
    "name": "Laptop",  
    "price": 50000  
}
```

You learned how dictionaries can represent structured information without creating a class.

Stage 8 — Functions

Functions allowed you to organize logic.

```
def greet(name):  
    print("Hello", name)
```

You learned:

- parameters
- arguments
- return values
- reusable logic

For example:

```
def calculate_total(a, b):  
    return a + b
```

Stage 9 — Error Handling

You learned that user input can cause errors.

Example:

```
try:  
    amount = float(input("Enter amount: "))  
except ValueError:  
    print("Invalid amount")
```

This is very useful in today's bank project.

Stage 10 — Files

You learned basic file writing:

```
with open("account.txt", "w") as file:  
    file.write("Account Summary")
```

and file reading:

```
with open("account.txt", "r") as file:  
    for line in file:  
        print(line.strip())
```

You also learned why `with` is useful: Python automatically closes the file.

Stage 11 — Classes and Objects

You moved into object-oriented programming.

You learned the basic structure:

```
class Student:
    def __init__(self, name):
        self.name = name
```

Then you created objects:

```
student1 = Student("Amit")
```

You learned that:

```
Class → blueprint
Object → actual thing created from blueprint
```

Stage 12 — Methods

You added behavior inside classes.

For example:

```
class Counter:
    def increase(self):
        # update value
```

Then, in your banking exercises, you practiced ideas like:

```
deposit()
withdraw()
get_balance()
```

Stage 13 — Basic Encapsulation

You practiced storing balance like:

```
self._balance
```

instead of freely modifying it everywhere.

The idea was:

```
Outside code
  ↓
BankAccount method
  ↓
Validate request
```

↓
Update _balance

Stage 14 — Product and Cart OOP

You also practiced OOP using:

Product
Cart

and methods such as:

add_product()
remove_product()
calculate_total()
search_product()

This helped you understand objects stored inside lists.

Stage 15 — Files + OOP Ideas

You then connected account information with files.

You practiced:

Bank/account data
↓
Convert to text
↓
Write file

and:

Text file
↓
Read
↓
Display account summary

Stage 16 — Testing and Debugging

Yesterday, on Day 59, you practiced manually testing programs.

You learned to think about:

Input
↓
Expected result
↓
Actual result
↓
Compare

You also considered edge cases such as:

```
Wrong password
Negative amount
Insufficient balance
```

Today you will use all of that thinking in one mini project.

5. Important Topics for Today

Your final project mainly combines:

```
variables
input/output
type conversion
if-elif-else
strings
lists
functions
error handling
classes
objects
methods
```

You also already understand:

```
loops
dictionaries
files
```

Even though the final project is intentionally small, those topics are now part of your foundation.

6. Foundational Notes

A. Keep the project small

Do not try to build:

- multiple bank accounts
- database storage
- login attempts
- transaction history files
- interest calculation
- account transfers
- ATM simulation

Those can come later.

Today's application should perform **one login and one banking action**.

B. Start with a balance

For example, conceptually:

```
starting balance = 5000
```

The program can create this internally.

The user does not need to enter the starting balance.

C. Check the password first

Your program should follow this order:

```
graph TD
    A[Ask password] --> B[Is password correct?]
    B -- / --> C[No]
    B -- \ --> D[Yes]
    C --> E[Stop]
    D --> F[Show menu]
```

Do not show the banking menu when the password is wrong.

D. Only one action

After successful login, display something like:

```
1. Check Balance
2. Deposit
3. Withdraw
```

The user chooses **one**.

After that action completes, the program can finish.

You do **not** need a repeated menu loop.

E. Validate money before changing balance

For deposit:

```
amount > 0
```

For withdrawal:

```
amount > 0
```

and:

```
amount <= balance
```

F. Transactions can be very simple

You do not need a complicated transaction system.

For example:

```
transactions = []
```

After a successful deposit, conceptually you might add:

```
"Deposited 500"
```

After a successful withdrawal:

```
"Withdrew 300"
```

You only need to save **one transaction** for this project.

G. Invalid operations should not change the balance

For example:

```
Balance = 5000  
Withdraw = 7000
```

Result:

```
Insufficient balance
```

Balance should still be:

```
5000
```

7. Easy Example

Before building the bank app, think about a much simpler example: a wallet.

Suppose:

```
Wallet balance = ₹1000  
Action = deposit  
Amount = ₹200
```

Think through it:

```
Current balance  
  ₹1000  
  +  
Deposit ₹200  
  ↓  
New balance  
  ₹1200
```

The important Python idea is:

```
Read value
  ↓
Validate value
  ↓
Update existing value
  ↓
Display result
```

The bank project uses exactly the same thinking.

8. Final Problem Statement

Final Beginner Python Mini Project — Simple Bank App

Create a very simple bank application.

The program should:

1. Have a predefined password.
2. Have a starting account balance.
3. Ask the user to enter the password.
4. If the password is wrong, display an error message and stop.
5. If the password is correct, show this menu:

```
1. Check Balance
2. Deposit
3. Withdraw
```

6. Allow the user to select **only one action**.
7. For **Check Balance**, display the current balance.
8. For **Deposit**, ask for an amount.
9. Deposit only if the amount is valid and greater than 0.
10. For **Withdraw**, ask for an amount.
11. Withdraw only if the amount is positive and the balance is sufficient.
12. Save one successful deposit or withdrawal transaction in a list.
13. Display the result.
14. End the program.

Keep everything simple.

9. Concepts Used

Variables

For things like:

```
password
balance
```

```
choice
amount
```

Input

For:

```
password
menu choice
deposit amount
withdrawal amount
```

Output

Using:

```
print()
```

to show messages.

Type Conversion

Convert monetary input from string to a number.

Conceptually:

```
"500"
↓
500.0
```

if-elif-else

Used for:

```
password validation
menu choice
amount validation
balance validation
```

Strings

Used for:

```
password
messages
transactions
```

Lists

Used for:

```
transactions
```

Functions

You could separate operations into functions such as:

```
deposit  
withdraw  
check balance
```

Error Handling

Useful when the user enters:

```
hello
```

instead of:

```
500
```

for an amount.

Classes and Objects

With the preferred OOP approach, you can create:

```
BankAccount class
```

and one:

```
BankAccount object
```

10. Thought Process

Before writing Python, solve the problem logically.

Step 1 — What data does the account need?

At minimum:

```
balance  
transactions
```

You may also keep the password separately.

Step 2 — What should happen first?

Authentication.

User enters password

Compare it with the correct password.

Step 3 — What happens if authentication fails?

Nothing else.

```
Wrong password
↓
Display message
↓
Program ends
```

Step 4 — What happens after successful login?

Display the menu.

```
1 → balance
2 → deposit
3 → withdraw
```

Step 5 — What information does each action need?

Check balance:

No additional input

Deposit:

deposit amount

Withdraw:

withdrawal amount

Step 6 — What must be validated?

Deposit:

```
Is amount a number?
Is amount > 0?
```

Withdrawal:

```
Is amount a number?
Is amount > 0?
Is amount <= balance?
```

Step 7 — When should a transaction be stored?

Only after a **successful financial operation**.

For example:

```
deposit successful
  ↓
update balance
  ↓
add transaction
```

Not:

```
invalid deposit
  ↓
add transaction ❌
```

11. Beginner-Friendly Pseudocode

Do not write Python immediately.

First write something similar to this:

```
START

Create correct password

Create bank account
  starting balance
  empty transactions list

Ask user for password

IF password is wrong
  print "Wrong password"

ELSE
  display menu

  ask user for choice

  IF choice is check balance
    display balance

  ELSE IF choice is deposit
    ask for amount

    try to convert amount to number

    IF amount is greater than zero
      add amount to balance
      save deposit transaction
```

```

        display updated balance
    ELSE
        display invalid amount

    ELSE IF choice is withdraw
        ask for amount

        try to convert amount to number

        IF amount is not positive
            display invalid amount

        ELSE IF amount is greater than balance
            display insufficient balance

        ELSE
            subtract amount from balance
            save withdrawal transaction
            display updated balance

    ELSE
        display invalid menu choice

END

```

That is already most of the problem-solving work.

The Python syntax comes afterward.

12. Suggested Solving Approach

Preferred: OOP-Based Approach

Because you recently learned classes and objects, this is a good final revision exercise.

Think about this structure:

```

BankAccount
|
├── _balance
├── transactions
|
├── get_balance()
├── deposit(amount)
└── withdraw(amount)

```

Then your main program handles:

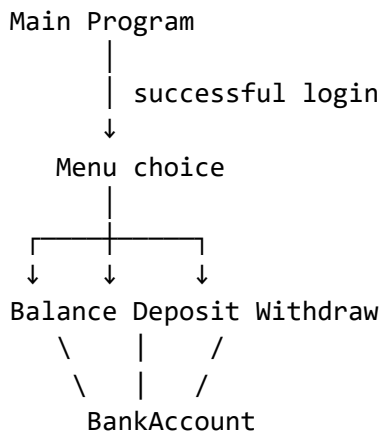
```

password
↓
menu
↓
user choice

```

↓
call object method

Conceptually:



This is the recommended approach.

Functional Approach

You could also solve it using variables and functions.

Conceptually:

```
balance
transactions

check_balance()
deposit()
withdraw()
```

This is also valid.

But for Day 60, try OOP first because it revises more of your recent learning.

13. Easy Edge Cases

Edge Case 1 — Wrong Password

Input:

Password: 9999

Expected behavior:

Wrong password

Do not show the bank menu.

Edge Case 2 — Invalid Deposit Amount

Example:

Deposit amount: -500

Expected:

Invalid amount

Balance should not change.

Edge Case 3 — Zero Amount

Deposit amount: 0

or:

Withdrawal amount: 0

Treat it as invalid.

Edge Case 4 — Non-Numeric Amount

Example:

Deposit amount: abc

The program should handle the conversion problem rather than crash.

Think about:

```
try:
    ...
except ValueError:
    ...
```

Edge Case 5 — Withdrawal Greater Than Balance

Suppose:

Balance = ₹5000
Withdrawal = ₹7000

Expected:

Insufficient balance

Balance remains ₹5000.

Edge Case 6 — Empty Transaction List

At the beginning:

```
transactions = []
```

That is perfectly valid.

If the user selects only **Check Balance**, it can remain empty.

Edge Case 7 — Invalid Menu Choice

Example:

```
Choice: 7
```

Expected:

```
Invalid choice
```

No balance changes should happen.

14. Common Mistakes to Avoid

Mistake 1 — Forgetting type conversion

This:

```
input()
```

returns a string.

For money calculations, you need a number.

Mistake 2 — Depositing negative amounts

Without validation, something like:

```
deposit -500
```

could incorrectly reduce the balance.

Check:

```
amount > 0
```

first.

Mistake 3 — Allowing overdrawing

Do not subtract the money before checking whether the account has enough balance.

Correct thinking:

```
Check balance first
```

↓

```
Enough?
```

/

```
No
```

↓

```
Reject
```

\

```
Yes
```

↓

```
Withdraw
```

Mistake 4 — Updating balance before validation

Avoid thinking like:

```
change balance
```

↓

```
check if valid
```

Instead:

```
validate
```

↓

```
change balance
```

Mistake 5 — Forgetting self

Inside an OOP method, account attributes usually need:

```
self.
```

Think:

```
self._balance
```

```
self.transactions
```

Mistake 6 — Forgetting to initialize the transaction list

Each account object should have its own empty list when it is created.

Conceptually:

```
New BankAccount
```

↓

```
transactions = []
```

Mistake 7 — Adding failed transactions

Do not add something to the transaction list when:

```
deposit was invalid
```

```
withdrawal failed
```

```
password was wrong
```

Store only the successful transaction.

Mistake 8 — Building too much

Do not turn today's exercise into a 500-line project.

You only need:

```
1 login
1 account
1 menu
1 selected operation
0 or 1 transaction
```

That is enough.

15. Five Quick Self-Check Questions

1. What does `input()` return by default?

Think about whether it returns an integer, float, or string.

2. Why should the deposit amount be checked before updating the balance?

Think about:

```
0
negative numbers
invalid values
```

3. Why do we check the balance before performing a withdrawal?

What would happen if:

```
Balance = 1000
Withdrawal = 5000
```

4. What is the difference between a class and an object?

Think:

```
BankAccount → ?
my_account → ?
```

5. When should a transaction be added to the transaction list?

Before validation, or only after a successful operation?

16. Hint Only

Start with the smallest possible class:

```
BankAccount
    balance
    transactions
```

Then think about three simple methods:

```
get_balance()
deposit(amount)
withdraw(amount)
```

Keep password checking and menu selection **outside the class** initially.

Build in this order:

```
BankAccount class
    ↓
Create account object
    ↓
Ask password
    ↓
Check password
    ↓
Show 3-option menu
    ↓
Read ONE choice
    ↓
Call appropriate method
    ↓
Save successful transaction
    ↓
End
```

For the money input, remember the pattern:

```
try
    convert input
    validate amount
    perform operation

except ValueError
    show invalid amount
```

And for withdrawal, remember the most important logic:

```
amount <= 0
    ↓
Invalid

amount > balance
    ↓
```

Insufficient balance

otherwise

↓

Withdrawal allowed

Your Day 60 goal is not complexity. Your goal is to independently connect Python fundamentals into one clean, working mini program without copying a complete solution.